

Machine Learning for Portfolio Construction

Group 1

Aya Talibi

Aya Errachidi

Mamadou Mbengue

Yacine Diop

Introduction

This project explores how we can use machine learning to build and optimize a simple investment portfolio made up of two stocks: Tesla (TSLA) and Apple (AAPL). Our main goal is to maximize the portfolio's Sharpe ratio and identify the efficient frontier, which represents the set of portfolios that offer the highest expected return for a given level of risk. In other words, the efficient frontier shows the most optimal combinations of risk and return that an investor can achieve through a certain asset allocation. The Sharpe ratio, which measures how much return an investor earns for each unit of risk taken, serves as an indicator to evaluate and compare portfolio performance along this frontier. By simulating thousands of different portfolio allocations between these two assets, we aim to find the combinations that offer the best trade-off between return and volatility, essentially, the most efficient portfolios.

To carry out this analysis effectively, it is essential to understand the nature of the two assets included in our portfolio: Tesla and Apple. Tesla is a global leader in electric vehicles and renewable energy, known for its rapid growth and high volatility. Apple, on the other hand, is one of the world's most valuable companies, offering strong financial stability through its consumer electronics and digital services. Together, these two assets provide a useful contrast: growth (more volatility) versus consistency (less volatility).

In this model we used historical adjusted closing prices from January 1, 2020, to the present date, which were directly retrieved through Yahoo Finance using the quantmod package in R. Yahoo Finance sources its market data from a combination of public financial exchanges (such as NASDAQ and NYSE) and third-party data providers, including companies like Morningstar and

ICE Data Services. While it is not considered an official source for institutional trading, it is widely used in academic and personal finance projects due to its broad coverage, free access, and seamless integration with tools like R through the quantmod package.

To reduce complexity and avoid overfitting, where the model might capture noise rather than meaningful patterns, we only used data from Wednesdays. Unlike other days of the week which often reflect reactions to weekend news or week-end positioning, Wednesdays typically don't coincide with market openings or closings, making them a neutral reference point for weekly performance.

To evaluate portfolio performance, we first simulated thousands of different allocations between Tesla and Apple using historical return data. Instead of calculating the Sharpe ratio manually for each one, we trained a Random Forest model to estimate it based on the asset weights. This helped us quickly spot which portfolios offered the best trade-off between risk and return. Using machine learning made the process faster and more practical, especially when dealing with a large number of simulations.

To conduct this analysis, we used various R packages for data collection, financial calculations, modeling, and visualization:

- quantmod: to download and extract historical stock data
- TTR: to calculate weekly returns from price data
- PerformanceAnalytics: for return, volatility, and Sharpe ratio computations
- randomForest: to model and predict Sharpe ratios based on portfolio weights
- ggplot2: to visualize the efficient frontier

Definitions

What is a portfolio, you might ask? A **portfolio** is a collection of financial investments owned by investors, which can either be an institution or an individual. Its main purpose is to balance risk and return depending on preferred outcomes.

In this paper, we are going to focus on stock (company stocks) portfolio optimization. Using mathematical equations, a portfolio's value is calculated as follows:

$$V = \sum_{i=1}^n P_i Q_i$$

where P_i represents the price per share of the i -th stock, and Q_i is the quantity of shares held. (Curry, 2025)

When building a portfolio, it is important to take time to evaluate two important things: how much risk you are willing to take and what return you are aiming for. This is where the **risk return tradeoff** theory appears.

As intuitive as this may sound, if you want to earn a high return, you generally should be comfortable taking a high risk (*no risk, no reward*). Similarly, choosing a safer option means accepting small gains. These two components are highly correlated and should be adjusted based on personal preferences. ("What Does Risk Return Tradeoff Tell Investors?", n.d.)

Since the main goal is to optimize the portfolio's value, the method we are using in this study is **machine learning (ML)**. ML involves exploiting algorithms that learn patterns and rules from a *labeled dataset*, which is a collection of examples where the answers are already known. The

more data we provide to these algorithms, the better they usually become at making accurate predictions.

The primary objective of an ML algorithm is to learn a model from this data so that it can correctly predict outcomes it has not encountered before. (Rebala, Ravi, & Churiwala, 2019)

The algorithm we are using is called **Random Forest**, which grows and combines multiple decision trees to create a “forest.” It starts at a single point and then branches off into two or more directions, with each branch of the decision tree offering different possible outcomes. (Liu, Wang, & Zhang, 2012)

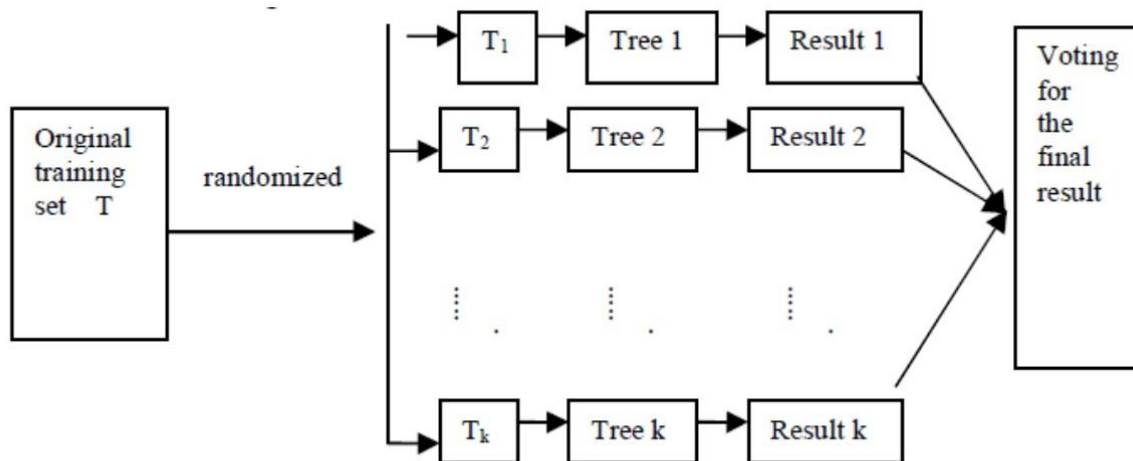


Fig. 1. Random forest schematic

The idea behind the **Random Forest** model is that combining a group of decision trees that differ from one another leads to better results than using a single tree on its own.

For *classification* tasks, each tree makes a prediction (similar to casting a vote), and the final prediction is the one that receives the most votes. For *regression*, instead of voting, each tree

outputs a numerical value, and the Random Forest takes the average of all those values to produce the final prediction. (Biau & Scornet, 2016)

Mathematical Model: Markowitz Theory

Historically, when we mention portfolio optimization, the first theory that comes to mind is the **Markowitz theory**. Developed by Harry Markowitz in 1952, this theory evaluates whether a portfolio is “good” based on the *mean* and *variance* of the returns of the assets within the portfolio.

His model relies on a few key assumptions:

1. The return from an investment is sufficient to describe how well it performs, and investors think of returns in terms of probability distributions.
2. When investors consider risk, they measure it using the **variance** — that is, how spread out the returns are.
3. Investors base their decisions solely on two factors: the *expected return* and the *variance* (risk) of that return.
4. Investors are risk-averse: if given two portfolios with the same expected return, they will choose the one with lower risk. Similarly, if two portfolios have the same risk, they will choose the one with the higher expected return. (Witt & Dobbins, 1979)

Let the expected return of the portfolio be defined as:

$$\mu = \mathbb{E}[R] = \sum_{i=1}^n \mu_i x_i$$

And the variance of the portfolio return as:

$$\sigma^2 = \sum_{i=1}^n \sum_{j=1}^n \sigma_{i,j} x_i x_j$$

The minimization problem is then reduced to:

$$\min(\sigma^2 - A\mu) \quad \text{subject to:} \quad \sum_{i=1}^n x_i = 1$$

Where A is called the **risk aversion index**. When $A = 0$, the investor only cares about minimizing risk, so the portfolio will have the smallest possible variance.

As A increases, it means the investor is more open to taking on risk in order to get a higher expected return. If A becomes large (approaching infinity), it indicates that the investor doesn't care about risk at all and just wants the highest return, no matter how risky the portfolio is. (Witt & Dobbins, 1979)

Although we do not explicitly solve the Markowitz optimization problem, our simulation-based approach approximates the **efficient frontier** (the set of optimal portfolios) by evaluating thousands of potential portfolio combinations.

Explanation of the Code / Our Model

To implement our portfolio optimization model, we first retrieved historical adjusted closing prices for Apple (AAPL) and Tesla (TSLA) using the quantmod package, focusing only on Wednesdays. We then calculated weekly returns, from which we derived the mean return vector and the covariance matrix — essential for understanding the expected performance and risk of each asset.

Using these statistics, we simulated 5,000 random portfolios by generating different combinations of weights between AAPL and TSLA. For each portfolio, we computed the annualized return, volatility, and the Sharpe ratio, which quantifies risk-adjusted performance.

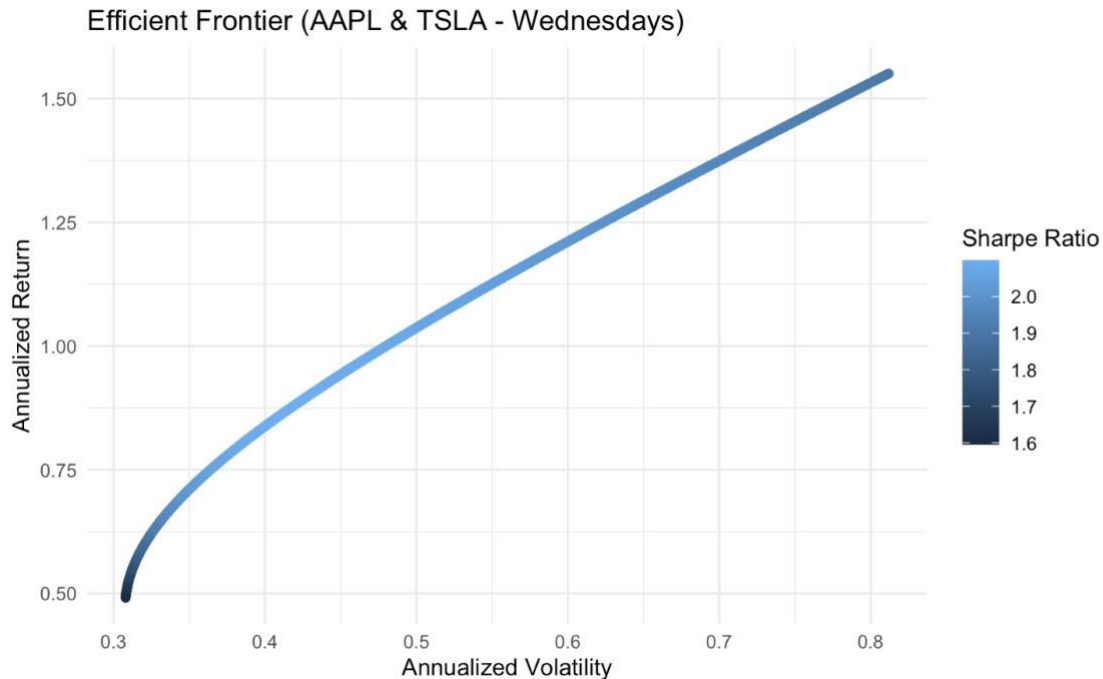
These results were compiled into a dataset used to train a Random Forest regression model (randomForest package), with the goal of predicting the Sharpe ratio based solely on asset weights. We split the data into 80% for training and 20% for testing, allowing us to evaluate how well the model performs on new, unseen portfolio configurations — achieving nearly perfect predictive accuracy (correlation ≈ 1) on the test set.

```
[1] "AAPL" "TSLA"  
Correlation (Test set): 0.9999975
```

Once trained, the model was used to predict the Sharpe ratios of 10,000 newly simulated portfolios, significantly accelerating the optimization process without recalculating risk and return manually. We then identified the portfolio with the highest predicted Sharpe ratio, indicating the most efficient trade-off between risk and return.

	AAPL <dbl>	TSLA <dbl>	predicted_sharpe <dbl>
4335	0.6275999	0.3724001	2.099093

Finally, we visualized all simulated portfolios using a scatter plot generated by ggplot2, where each point represents a portfolio, with volatility on the x-axis, return on the y-axis, and color intensity reflecting the Sharpe ratio. This graph shows the efficient frontier, helping us see which portfolios offer the best performance for a given level of risk.



Risk and Return Trade-off

Efficient Frontier Analysis

The simulation of 10,000 portfolio allocations and the application of a Random Forest regression model enabled the empirical approximation of the efficient frontier for a two-asset portfolio composed of Tesla (TSLA) and Apple (AAPL). The resulting scatter plot displayed the characteristic convex shape predicted by Modern Portfolio Theory, representing the optimal trade-off between expected return and portfolio volatility. Portfolios lying along this frontier achieved superior Sharpe ratios, illustrating the set of most efficient risk-return combinations, while those positioned below the frontier were identified as suboptimal.

Key Observations

An analysis of the portfolio allocations revealed a distinct pattern: portfolios with a heavier weighting towards Tesla exhibited elevated levels of both return and volatility, reflecting Tesla's inherent high-growth, high-risk nature. Conversely, portfolios skewed towards Apple demonstrated reduced volatility and moderate returns, consistent with Apple's established role as a stable and defensive stock. The most efficient portfolios, those maximizing the Sharpe ratio, exhibited a slight overweight towards Apple, indicating that in this context, managing volatility contributed more significantly to enhancing risk-adjusted returns than aggressively pursuing higher nominal returns.

Random Forest Model Performance

The Random Forest regression model demonstrated exceptional predictive performance, achieving a near-perfect correlation between the predicted and actual Sharpe ratios in the test set. This strong predictive accuracy suggests that the model effectively captured the non-linear and complex relationships between portfolio weights and their corresponding risk-adjusted returns. The application of machine learning also significantly reduced computational time by enabling the rapid prediction of Sharpe ratios across thousands of portfolio configurations without the need to manually recalculate return and volatility for each simulation.

Discussion

The successful approximation of the efficient frontier and identification of optimal portfolios underscore the power of machine learning techniques in modern portfolio optimization. Unlike traditional parametric approaches such as Markowitz's mean-variance optimization — which

rely heavily on assumptions of linearity and normally distributed returns — the Random Forest model adapts flexibly to more intricate, real-world data structures. This project thus illustrates how data-driven models can provide superior practical outcomes, especially when handling the complex dependencies that characterize financial markets.

Risk and Return Trade-off Summary

The results reaffirmed the fundamental principle of the risk-return trade-off: higher returns are generally accompanied by higher levels of risk. However, the analysis also highlighted that carefully balancing asset allocations could lead to significant improvements in risk-adjusted returns without necessitating a proportionate increase in risk. By favoring slightly more stable assets like Apple, investors could achieve portfolios positioned closer to the optimal frontier, maximizing the Sharpe ratio and securing more efficient risk-return profiles.

Conclusion

This project demonstrated that machine learning, particularly Random Forests, can effectively optimize simple portfolios by predicting Sharpe ratios from simulated allocations. While the approach proved promising with two assets, it also highlighted important challenges as the number of assets increases such as the risk of overfitting, increased computational demands, and the curse of dimensionality. Additionally, the analysis was limited by its narrow focus on two technology stocks and reliance on historical data, which restricted diversification and ignored important real-world considerations like future market expectations or transaction costs.

Looking ahead, future iterations of this model could benefit from incorporating forward-looking indicators, a broader and more dynamic set of asset classes, and alternative machine learning methods such as reinforcement learning. These enhancements would allow the model to better capture market complexity, improve decision-making under uncertainty, and move closer to applications in real-world portfolio management.

References

Curry, B. (2025, March 3). *What is a portfolio?* Forbes Advisor. <https://www.forbes.com/advisor/investing/portfolio/>

What does risk return tradeoff tell investors? (n.d.). SmartAsset. <https://smartasset.com/investing/risk-return-tradeoff>

Rebala, G., Ravi, A., & Churiwala, S. (2019). Machine Learning Definition and Basics. In *Springer eBooks* (pp. 1–17). https://doi.org/10.1007/978-3-030-15729-6_1

Liu, Y., Wang, Y., & Zhang, J. (2012). New machine learning algorithm: random Forest. In *Lecture notes in computer science* (pp. 246–252). https://doi.org/10.1007/978-3-64234062-8_32

Biau, G., & Scornet, E. (2016). A random forest guided tour. *Test*, 25(2), 197– 227. <https://doi.org/10.1007/s11749-016-0481-7>

Witt, S. F., & Dobbins, R. (1979). The Markowitz Contribution to Portfolio Theory. *Managerial Finance*, 5(1), 3–17. <https://doi.org/10.1108/eb013433>

Stephen F. Witt, Richard Dobbins. (1979) The Markowitz Contribution to Portfolio Theory. *Managerial Finance*, Vol. 5 Iss: 1 pp. 3 - 17. DOI: 10.1108/eb013433

Appendix

```
# The necessary packages:
```

```
library(quantmod)
```

```
library(tibble)
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(randomForest)
```

```
# The stock data symbols <- c("AAPL", "TSLA")
```

```
getSymbols(symbols, from = "2020-01-01", to = "2022-01-01")
```

```
## [1] "AAPL" "TSLA"
```

```
# Extraction of adjusted prices
```

```
prices <- merge(Ad(AAPL), Ad(TSLA))
```

```
colnames(prices) <- symbols
```

```
wednesday_prices <- prices[weekdays(index(prices)) == "Wednesday"]
```

```
# Calculation of weekly returns
```

```
returns <- na.omit(ROC(wednesday_prices, type = "discrete"))
```

```
mean_returns <- colMeans(returns) cov_matrix <- cov(returns)
```

```
# Portfolio simulation
```

```
set.seed(42)
```

```
n_portfolios <- 5000
```

```
results <- matrix(nrow = n_portfolios, ncol = 3) weights_list  
<- list()
```

```
for (i in 1:n_portfolios) { weights  
<- runif(length(symbols)) weights  
<- weights / sum(weights)
```

```

weights_list[[i]] <- weights

port_return <- sum(weights * mean_returns) * 52
port_volatility <- sqrt(t(weights) %*% cov_matrix %*% weights) * sqrt(52)
sharpe_ratio <- port_return / port_volatility
  results[i, ] <- c(port_return, port_volatility,
sharpe_ratio)
}

# Data set for machine learning
weights_df <- as.data.frame(do.call(rbind,
weights_list)) colnames(weights_df) <- symbols results_df
<- as.data.frame(results)
colnames(results_df) <- c("Return", "Volatility", "Sharpe")
data_rf <- cbind(weights_df, results_df)

# Training of Random Forest:
# We split the dataset into training (80%) and testing (20%) sets
set.seed(123)
train_idx <- sample(1:nrow(data_rf), 0.8 *
nrow(data_rf)) train_data <- data_rf[train_idx, ]
test_data <- data_rf[-train_idx, ]

rf_model <- randomForest(Sharpe ~ AAPL + TSLA, data = train_data, ntree = 500
)

# Evaluation of the model on the test data
test_data$predicted_sharpe <- predict(rf_model, newdata = test_data)
cat("Correlation (Test set):", cor(test_data$Sharpe, test_data$predicted_shar
pe), "\n")

## Correlation (Test set): 0.9999976

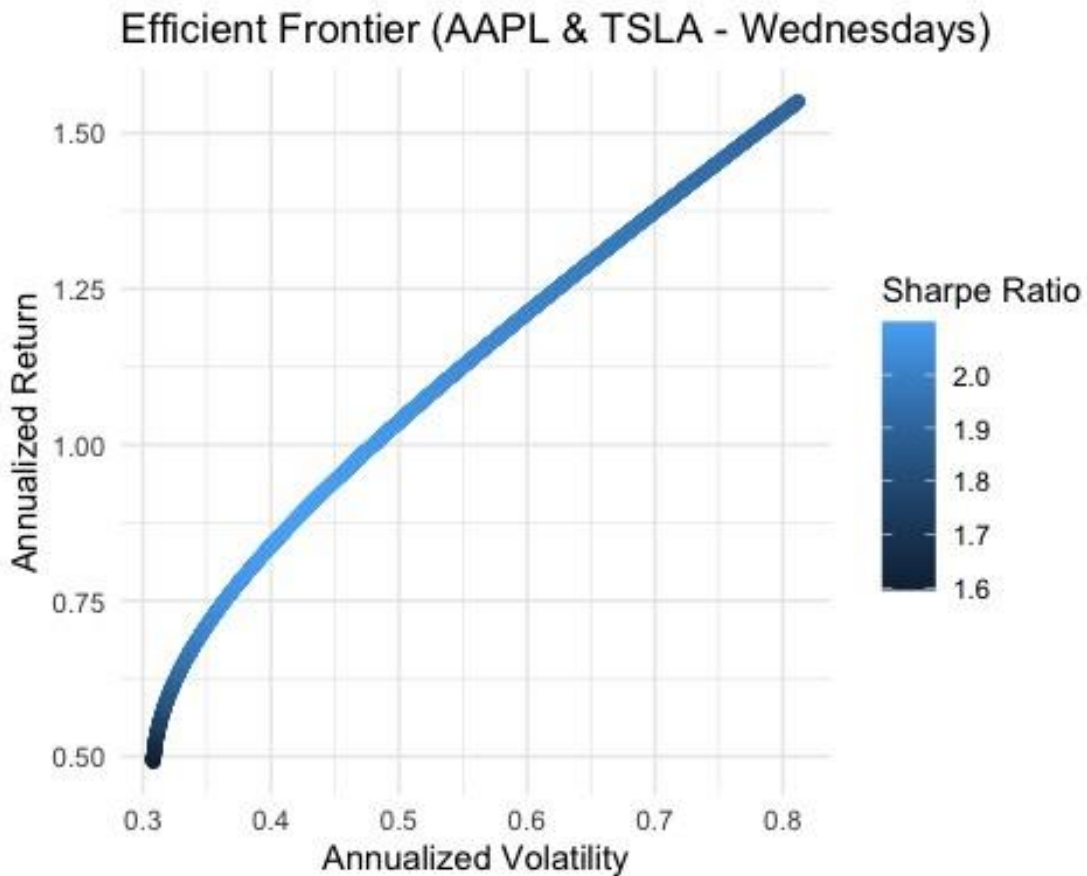
# Generation of new portfolios and predict their Sharpe ratios using the trai
ned model
new_weights <- matrix(runif(10000 * length(symbols)), ncol = 2)
new_weights <- t(apply(new_weights, 1, function(w) w / sum(w)))
colnames(new_weights) <- symbols
new_weights_df <- as.data.frame(new_weights)
new_weights_df$predicted_sharpe <- predict(rf_model, newdata = new_weights_df
)

```

```
# Identification of the portfolio with the best predicted Sharpe ratio
best_idx <- which.max(new_weights_df$predicted_sharpe) best_weights <-
new_weights_df[best_idx, ]

ggplot(data_rf, aes(x = Volatility, y = Return, color = Sharpe)) +
geom_point(alpha = 0.6) +
  labs(title = "Efficient Frontier (AAPL & TSLA - Wednesdays)",
```

```
x = "Annualized Volatility", y = "Annualized Return", color = "Sharpe Ratio") + theme_minimal()
```



```
cat("Best predicted weights:\n")  
## Best predicted weights:  
print(best_weights)  
## AAPL TSLA predicted_sharpe  
## 1447 0.6275925 0.3724075 2.099093
```